Отчёт о работе: Промпт-инжиниринг и основы RAG

Дисциплина: Методы искусственного интеллекта I (ТюмГУ, 5 семестр) **Автор:** Макеев Глеб Борисович · **Дата:** 04.11.2026 **Задание:** практика «Промпт-инжиниринг и основы RAG» (открыто 30.10.2026, срок сдачи 20.01.2027)

0. Коротко

Что	Результат
База знаний	1 документ, 4 абзаца о вымышленной компании «МетроСервис» (поверка средств измерений)
Чанки	**12** (сплиттер `RecursiveCharacterTextSplitter`, 500 токенов / 50 перекрытие)
Эмбеддинги	`intfloat/multilingual-e5-small` (sentence-transformers, CPU)
Векторное хранилище	**Chroma** (`hnsw:space = cosine`, persist в `chroma/`)
Цепочка	retriever **top-3** → строгий промпт → LLM (LCEL: `prompt` → `llm` → `StrOutputParser`)
LLM	`deepseek/deepseek-v4.1-flash` через OpenAI-совместимый API RouterAI
Проверка	3 вопроса с ответом в базе + 1 контрольный (ответа нет) → **корректный отказ**
Задание со звёздочкой	реализовано: печатаются исходные чанки и их релевантность

Время ответа: 1,2–3,2 с на вопрос (эмбеддинг запроса + генерация), полностью воспроизводимо на CPU без GPU.

1. Постановка (по чек-листу задания)

Нужно было собрать классический RAG-конвейер из двух этапов:

1. **Ingestion** — загрузка документов (текст/PDF/DOCX), разбиение на чанки (≈500 токенов с перекрытием 50), векторизация эмбеддинг-моделью, запись векторов и исходного текста в векторную БД.
2. **Chain** — связывание компонентов через LangChain: `Retriever` (топ-3 похожих чанка), шаблон промпта `[системная инструкция] + [найденные чанки] + [вопрос]`, `LLM`. Смысл LangChain в задании подчёркнут отдельно: LLM должна заменяться без переписывания логики.

Практический чек-лист: зависимости, база знаний на 3–4 абзаца, `RecursiveCharacterTextSplitter` + `Chroma.from_documents`, строгий промпт против выдумывания, цепочка (`RetrievalQA` или кастомная), 2–3 вопроса и проверка отказа на вопрос, которого нет в тексте.

2. База знаний

Взята **вымышленная организация** — ЗАО «МетроСервис», аккредитованная поверочная лаборатория (Тюмень, Сургут, Ноябрьск): профиль, услуги и парк средств измерений, порядок работ и сроки, гарантии, скидки и контакты. Файл `knowledge_base.md`, 4 абзаца.

Почему вымышленная: задание разрешает любой вариант, а вымышленная компания даёт полный контроль над фактами — можно заранее знать, какие вопросы имеют ответ, а какие обязаны привести к отказу. Тематика выбрана по профессии (метрология и АСУ ТП), чтобы проверить прототип на близком мне словаре: расходомеры, СИКН, ФГИС «Аршин», свидетельство о поверке.

3. Этап 1: подготовка базы знаний

```
docs = TextLoader("knowledge_base.md", encoding="utf-8").load()
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=500, chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " "],
)
chunks = splitter.split_documents(docs)
embeddings = E5Embeddings(model_name="intfloat/multilingual-e5-small")
store = Chroma.from_documents(
    documents=chunks, embedding=embeddings,
    collection_name="metroservice",
    collection_metadata={"hnsw:space": "cosine"},
    persist_directory="chroma",
)
```

Получилось **12 чанков** — в основном по абзацу, крупные абзацы разбиты с перекрытием:

Чанк	Символов	Начало
1	76	`# База знаний: ЗАО «МетроСервис» …`
2	422	«ЗАО «МетроСервис» — аккредитованная поверочная лаборатория…»
3	117	«систем измерений количества и показателей качества нефти (СИКН)…»
4	32	`## 2. Услуги и парк оборудования`
5	448	«Компания выполняет поверку и калибровку средств измерений…»
6	64	«стендах в Тюмени и Сургуте, так и с выездом на объект заказчика…»
7	28	`## 3. Порядок работы и сроки`
8	443	«Заявка на поверку оформляется по электронной почте…»
9	177	«Сведения о результате поверки вносятся в ФГИС «Аршин»…»
10	36	`## 4. Гарантии, стоимость и контакты`
11	437	«Гарантия на выполненные поверочные работы — 12 месяцев…»
12	196	«телефон +7 (3452) 55-12-34, почта zakaz@metroservice.example…»

Модель эмбеддингов. Первая версия была на `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` с косинусной метрикой. На вопросе «какие документы получает заказчик и куда вносится результат» она подняла в топ-3 нерелевантные чанки (гарантия, услуги), и модель честно ответила «в базе нет информации», хотя нужный абзац про ФГИС «Аршин» в базе был. Замена на `intfloat/multilingual-e5-small` (с обязательными префиксами `query:` / `passage:`) и косинусное расстояние исправили выдачу — нужный чанк стал третьим с релевантностью 0.856.

4. Этап 2: RAG-цепочка

```
hits = store.similarity_search_with_relevance_scores(question, k=3)
context = "\n\n".join(f"[Ф{i}] (релевантность {score:.3f})\n{doc.page_content}"
                      for i, (doc, score) in enumerate(hits, start=1))
answer = (PROMPT | llm | StrOutputParser()).invoke({"context": context, "question":
question})
```

`ChatOpenAI(model="deepseek/deepseek-v4.1-flash", base_url="https://routerai.ru/api/v1", temperature=0)` — OpenAI-совместимый клиент, поэтому замена модели (на локальную или YandexGPT) сводится к смене двух параметров: ровно то преимущество LangChain, о котором говорится в задании.

Промпт (строгий, запрещает выдумывать):

```
system: Ты — ассистент службы метрологии. Отвечай ТОЛЬКО на основании фрагментов
контекста.
        1. Не используй знания вне контекста, не додумывай факты и числа.
        2. Если в контексте нет ответа, ответь ровно так: «В базе знаний нет информации
об этом.»
        3. Отвечай кратко, 1–3 предложениями, можно цитировать числа из контекста.
        4. В конце ответа укажи, на какие фрагменты ты опирался: [Ф1], [Ф2].

human:   Контекст: {context}

         Вопрос: {question}
```

Четыре правила решают четыре типовые проблемы RAG: (1) запрет на внешние знания отсекает галлюцинации, (2) явная фраза-отказ делает поведение проверяемым, (3) ограничение длины не даёт модели «растекаться», (4) обязательные ссылки на фрагменты — это уже задание со звёздочкой: ответ можно проверить по цитате.

5. Результаты

Вопрос 1 — сроки (ответ есть в базе)

Вопрос: Сколько времени занимает поверка на стенде и поверка на месте эксплуатации?

Найденные чанки: `[Ф1] 0.901` (заявка и сроки), `[Ф2] 0.857` (гарантия), `[Ф3] 0.834` (ФГИС «Аршин»).

> **Ответ:** Поверка на стационарном стенде выполняется за 2–3 рабочих дня с момента поступления средства измерения, а поверка на месте эксплуатации — за один рабочий день, если партия не превышает 20 средств измерений [Ф1].

Вопрос 2 — документы и ФГИС «Аршин» (ответ есть в базе)

> **Вопрос:** Какие документы получает заказчик и куда вносится результат поверки?

Найденные чанки: `[Ф1] 0.873` (заявка и документы), `[Ф2] 0.859` (гарантия), `[Ф3] 0.856` (ФГИС «Аршин»).

> **Ответ:** Заказчик получает свидетельство о поверке и протокол поверки с результатами измерений [Ф1]. Сведения о результате поверки вносятся в Федеральный информационный фонд по обеспечению единства измерений (ФГИС «Аршин») в течение пяти рабочих дней после подписания протокола [Ф3].

Ответ собран из **двух разных чанков** — это и есть демонстрация того, что RAG не пересказывает один абзац, а склеивает факты из разных мест базы.

Вопрос 3 — скидка (ответ есть в базе)

> **Вопрос:** При каком объёме работ дают скидку и сколько процентов?

Найденные чанки: `[Ф1] 0.838`, `[Ф2] 0.813`, `[Ф3] 0.803`.

> **Ответ:** При единовременном объёме от 50 средств измерений предоставляется скидка 10 % [Ф1].

Вопрос 4 — контрольный, ответа в базе НЕТ

> **Вопрос:** Какой ИНН у компании «МетроСервис»?

Найденные чанки: `[Ф1] 0.883`, `[Ф2] 0.871`, `[Ф3] 0.870` — все про компанию, но ИНН в них нет.

> **Ответ:** В базе знаний нет информации об этом.

Ключевая проверка пройдена: релевантность высокая (0.87+), потому что вопрос «про компанию», но конкретного факта нет — модель **отказалась отвечать**, а не придумала ИНН. Именно так работает правило 2 из промпта.

6. Задание со звёздочкой: релевантность и источники

Реализовано в `rag_demo.py`: для каждого вопроса печатаются топ-3 чанка с косинусной релевантностью (0…1) и первые 110 символов текста, а модель обязана ссылаться на номера фрагментов `[Ф1]`…`[Ф3]`. Пример из прогона:

```
Вопрос 2: Какие документы получает заказчик и куда вносится результат поверки?
  [Ф1] 0.873 — Заявка на поверку оформляется по электронной почте или через личный
кабинет…
  [Ф2] 0.859 — Гарантия на выполненные поверочные работы — 12 месяцев…
  [Ф3] 0.856 — Сведения о результате поверки вносятся в Федеральный информационный фонд…
```

Такой вывод позволяет за минуту понять, почему модель ответила именно так, и увидеть, что при неудачной модели эмбеддингов (первая версия) релевантные чанки в топ-3 просто не попадали.

7. Ограничения и что дальше

1. **База знаний маленькая (4 абзаца)** — этого достаточно для демонстрации, но не для проверки качества поиска: на 12 чанках «топ-3» покрывает четверть базы, поэтому почти любой вопрос находит хоть что-то похожее.
2. **Нет порога отсечения.** Сейчас решение «нет ответа» принимает LLM по промпту; надёжнее добавить порог по релевантности (например, отбрасывать чанки ниже 0,75) — но на этой базе скоры сжаты в диапазон 0,80–0,90, и порог подобрать нельзя без реальных документов.
3. **Одна модель эмбеддингов, без ре-ранжирования.** Следующий шаг — `bge-reranker` или cross-encoder поверх топ-20, это заметно поднимает точность на длинных нормативных текстах.
4. **Нет гибридного поиска.** Для ГОСТов и обозначений (`ФГИС «Аршин»`, `СИКН-3`) полезно добавить BM25 к векторному поиску: точные аббревиатуры векторный поиск «размывает».
5. **Цитаты не проверяются автоматически** — автоматически — модель может сослаться на фрагмент, где факта нет (в вопросе 3 формально так и было: скидка есть в [Ф1], но чанк назван «гарантия»). Лечится требованием цитировать дословную подстроку или автоматической проверкой вхождения ответа.
6. **Данные.** Все факты вымышлены; прототип не содержит персональных данных и коммерческой тайны.

8. Выводы

1. Классическая схема «чанки → эмбеддинги → векторная БД → топ-k → строгий промпт → LLM» собирается за один вечер и работает на CPU.
2. **Качество RAG определяется качеством поиска, а не моделью генерации.** Первый прогон с `paraphrase-multilingual-MiniLM` давал неверный ответ при наличии нужного абзаца в базе; смена только модели эмбеддингов исправила все три вопроса, не меняя промпт и LLM.
3. Строгий промпт с явной фразой-отказом даёт проверяемое поведение: на вопрос вне базы модель отвечает «в базе знаний нет информации», а не выдумывает факт.
4. LangChain-цепочка позволяет заменить LLM (облако ↔ локальная модель) без изменения логики — это и есть практическая ценность фреймворка для корпоративного контура.

9. Как воспроизвести

```
cd ~/projects/study/tsu/5sem/Методы\ исскуственного\ ителекта/Практика_4_RAG
uv venv --python 3.12 .venv
uv pip install torch --index-url https://download.pytorch.org/whl/cpu
uv pip install -r requirements.txt
source ~/.config/zsh/secrets.zsh   # ROUTERAI_API_KEY
.venv/bin/python rag_demo.py
```

Версии: Python 3.12.13, langchain 1.4.8, langchain-core 1.7.5, langchain-openai 1.7.3, langchain-huggingface 1.2.3, langchain-chroma 1.1.0, chromadb 1.5.9, sentence-transformers 6.2.0, transformers 5.22.1, torch 2.15.0+cpu, tiktoken 0.14.0.

Приложение А. Полный лог прогона

Полный вывод с чанками, вопросами, ответами и релевантностью — файл `rag_output.txt` (67 строк) в папке работы.

Приложение Б. Состав папки

Файл	Что это
`knowledge_base.md`	база знаний (4 абзаца о вымышленной компании)
`rag_demo.py`	весь конвейер: ingestion + цепочка + вывод источников
`requirements.txt`	зависимости
`chroma/`	сохранённое векторное хранилище
`rag_output.txt`	лог прогона (4 вопроса)